

WEEK 2 · DAY 1

IAM Deep Dive

— Roles vs Users

BaseStack Academy · AWS Cloud Accelerator

IAM & Security

Today's Agenda

IAM Deep Dive — Roles vs Users

1

Overview — Why This Matters

FOUNDATION

2

Key Concepts — Definitions & Vocabulary

CORE KNOWLEDGE

3

Deep Dive — Key Concepts Extended

DEEPER LEARNING

4

How It Works — Step by Step

MECHANICS

5

Technical Deep Dive

ADVANCED DETAIL

6

Architecture & Visual Model

VISUAL

7

Real-World Nigerian Scenario

CONTEXT

8

Common Mistakes & Exam Traps

EXAM PREP

Overview — Why This Matters

AWS Identity and Access Management (IAM) is the foundation of every AWS security posture. It is the service that controls who can do what in your AWS account. IAM is a global service — meaning IAM users, groups, roles, and policies you create are available across all Regions in your account without any additional configuration.

IAM is one of the most tested areas on the SAA-C03 exam. Domain 1 (Design Secure Architectures) accounts for 30% of the exam — and IAM is at the centre of that domain. Mastering IAM now means mastering the largest single exam domain.

WHY NOW?

Cloud skills are the #1 most in-demand tech capability globally. Understanding this topic is essential for both the SAA-C03 exam and real cloud engineering work.



Beginner Note

Do not worry if this feels new — every expert was once a complete beginner. Take notes and ask questions freely.



Key Concepts — Know These Definitions

📖 Study tip: Say each definition aloud — spoken repetition builds memory 3× faster than reading alone.

Root Account

The initial login created with your email — has unlimited power, NEVER use for daily operations

IAM User

An identity representing a person or application — has no permissions by default until you assign them

IAM Group

A collection of IAM users — permissions assigned to a group are inherited by all members

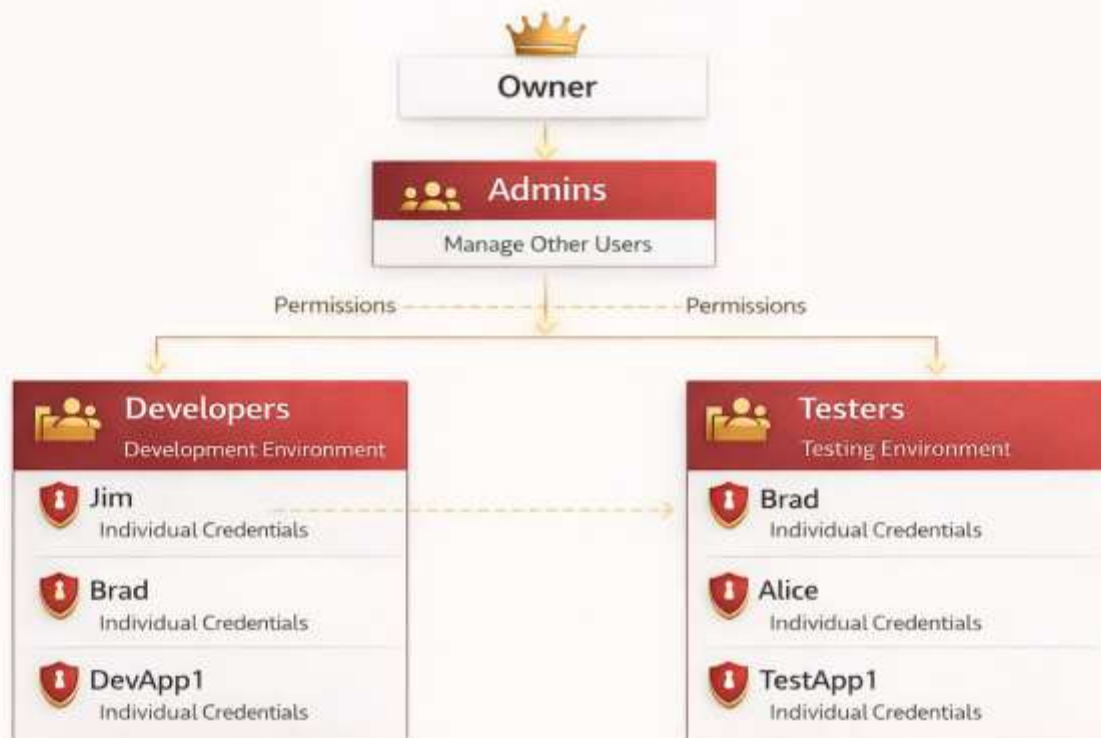
IAM Policy

A JSON document that defines permissions — attached to users, groups, or roles

Least Privilege

Grant only the minimum permissions required — nothing more, nothing less

IAM Groups Diagram



Permissions flow down from Groups to Users

Define permissions at the group level, inherited by all users in the group automatically



IAM Roles — The Other Identity

🔑 Key insight: A Role is NOT a user. It has no password or access keys. It is an identity you temporarily become.

IAM Role

A temporary identity assumed by AWS services, users, or applications — has no password or long-term access keys

Trust Policy

Attached to every Role — defines WHO is allowed to assume it (e.g., EC2 service, Lambda, another AWS account)

STS (Security Token Service)

The AWS service that issues temporary credentials when a role is assumed — credentials expire automatically after a set duration

AssumeRole

The API action used to request temporary credentials from STS — called by EC2, Lambda, or a user switching into a role

When to Use a Role

Use a Role (not a User) when an AWS service, application, or external account needs access to your resources

🔑 IAM Users vs IAM Roles — The Core Distinction

Users vs Roles — Know the Difference

This is the most exam-tested distinction in IAM. If you understand this table, you can answer most IAM scenario questions.

✓ IAM USER — Has permanent credentials (password + access keys). Used for human operators.

✓ IAM ROLE — Has NO permanent credentials. Issues temporary keys via STS. Used for services.

✓ USE A USER WHEN — A person needs to log into the AWS Console or run CLI commands daily.

✓ USE A ROLE WHEN — An EC2 instance, Lambda function, or another AWS service needs permissions.

⚠️ EXAM RULE — Never embed access keys in application code. Attach an IAM Role to the service instead.

⚙️ How It Works — Step by Step

1

Step 1 — Lock Down the Root Account. When you first create an AWS account, the root user has unlimited power — it can close the account, change billing, and bypass every restriction. Immediately: enable MFA on root, create a separate IAM admin user for daily work, and never use root again. This is AWS best practice and an exam certainty.

2

Step 2 — Create IAM Users for Human Access. Each person on your team gets their own IAM User with a unique username, password, and optionally access keys for CLI use. Add users to IAM Groups — a user inherits all permissions attached to their group. Never share credentials between people; one person = one IAM User.

3

Step 3 — Attach Roles to AWS Services. When an EC2 instance or Lambda function needs to access S3, do NOT create an IAM User for it. Instead, create an IAM Role with the required permissions and attach it to the service. The service uses STS to obtain temporary credentials automatically — no hardcoded keys, no security risk. IAM itself is free.

How IAM Roles Work Under the Hood

Every IAM Role has two policies attached to it. The Trust Policy answers: WHO can assume this role? For example, a trust policy on an EC2 role states that the `ec2.amazonaws.com` service principal is allowed to call `AssumeRole`. The Permissions Policy answers: WHAT can this role do once assumed? For example, it can read objects from a specific S3 bucket. These two policies work together — trust grants access, permissions define actions.

When an EC2 instance assumes a role, AWS STS (Security Token Service) issues temporary credentials — an access key ID, a secret access key, and a session token — that expire after a defined period (default 1 hour, max 12 hours for EC2). The application on the instance uses these credentials automatically via the instance metadata service at `169.254.169.254`. No developer ever touches these keys. This is why roles are far more secure than long-term access keys stored in application code.

? Architecture & Visual Model

Core Components

Root Account

Root Account — Created automatically. Lock with MFA immediately. Never use for daily work.

IAM User

IAM Users — Added to Groups. Groups carry the policies. Users inherit all group permissions automatically.

IAM Group

IAM Groups — Attach policies here, not to individual users. One policy change updates all members instantly.

Service Access (Roles)

IAM Policy

IAM Roles — Attached to AWS services (EC2, Lambda). STS issues temporary credentials. No hardcoded keys ever.

Least Privilege

IAM Policies — JSON documents attached to Users, Groups, or Roles. Define Effect, Action, and Resource for every permission.

Root Account

Email-based AWS account, MFA enabled, not used daily

IAM Users

Human identities with permanent credentials

IAM Groups

Admins, Developers, Analysts, Managers

IAM Roles

- **Trust Policy**
Who can assume the role
- **Permissions Policy**
What actions are allowed

IAM Policies

JSON permissions (Allow/Deny, Action, Resource)

AssumeRole

AWS STS

AssumeRole

Temporary Credentials

EC2 Instance

Lambda Function

S3 Bucket

CloudWatch

DynamoDB

Least Privilege

Grant only required access

? Architecture & Visual Model

Core Components

Root Account

Root Account — Created automatically. Lock with MFA immediately. Never use for daily work.

IAM User

IAM Users — Added to Groups. Groups carry the policies. Users inherit all group permissions automatically.

IAM Group

IAM Groups — Attach policies here, not to individual users. One policy change updates all members instantly.

Service Access (Roles)

IAM Policy

IAM Roles — Attached to AWS services (EC2, Lambda). STS issues temporary credentials. No hardcoded keys ever.

Least Privilege

IAM Policies — JSON documents attached to Users, Groups, or Roles. Define Effect, Action, and Resource for every permission.



Real-World Nigerian Scenario

A Lagos consulting firm has 20 staff: 5 developers, 10 data analysts, and 5 managers. The IT admin creates three IAM Groups — Developers (EC2 and Lambda access), Analysts (S3 and Athena access), and Managers (read-only across all services). Each staff member gets one IAM User added to their group. Adding a new hire takes 30 seconds. Removing a departing employee deletes one IAM User and all their access disappears instantly. The firm also runs EC2 instances that process client data stored in S3. Rather than embedding access keys in the code — which would be a critical security risk — the admin attaches an IAM Role to each EC2 instance with only the S3 permissions it needs. No developer ever sees or handles those credentials.

Discussion Question

A startup in Abuja is building a mobile banking app on AWS. Their backend runs on EC2 and needs to store transaction records in S3 and query a DynamoDB table. A junior developer suggests creating an IAM User with access keys and putting them in the application code. Is this the right approach? What should they do instead, and why?

⚠️ Common Mistakes & Exam Traps

- These are the most common exam errors — every one costs marks. Memorise them.



Root account CANNOT be restricted by any IAM policy. Nothing limits root — not even SCPs from AWS Organizations. This is why you lock it with MFA and never use it again.



IAM is GLOBAL — users, roles, and policies are not Regional. However, the resources they access (EC2, S3 buckets, RDS) are Regional or global depending on the service. Do not confuse IAM scope with resource scope.



NEVER hardcode access keys into application code or EC2 user data scripts. The exam will present this as an option — it is always wrong. The correct answer is always: attach an IAM Role to the EC2 instance or Lambda function instead.



Question

A developer at a Lagos fintech company has written a Python application deployed on an EC2 instance. The application needs to read files from an S3 bucket and write logs to CloudWatch. The developer has created an IAM User, generated access keys, and pasted them directly into the application code on the server. A senior engineer flags this as a security violation. What is the CORRECT solution?

A Encrypt the access keys using AWS KMS before storing them in the application code on the EC2 instance.

B Store the IAM User access keys in an environment variable file (.env) on the EC2 instance and restrict file permissions to the application user only.

C Remove the access keys from the code. Create an IAM Role with S3 read and CloudWatch write permissions, then attach the role to the EC2 instance. The application will retrieve temporary credentials automatically.

D Create a new IAM User specifically for the EC2 instance, assign it to a group with the required policies, and rotate the access keys every 30 days using a Lambda function.



Question

A developer at a Lagos fintech company has written a Python application deployed on an EC2 instance. The application needs to read files from an S3 bucket and write logs to CloudWatch. The developer has created an IAM User, generated access keys, and pasted them directly into the application code on the server. A senior engineer flags this as a security violation. What is the CORRECT solution?

- A** Encrypt the access keys using AWS KMS before storing them in the application code on the EC2 instance.
- B** Store the IAM User access keys in an environment variable file (.env) on the EC2 instance and restrict file permissions to the application user only.
- C** Remove the access keys from the code. Create an IAM Role with S3 read and CloudWatch write permissions, then attach the role to the EC2 instance. The application will retrieve temporary credentials automatically.
- D** Create a new IAM User specifically for the EC2 instance, assign it to a group with the required policies, and rotate the access keys every 30 days using a Lambda function.

✓ Answer: C — Attach an IAM Role to EC2. STS issues temporary credentials automatically.

💡 Key Takeaways — What You Learned Today

IAM Deep Dive — Roles vs Users

✓ Concepts Covered

- ✓ Root Account — lock it, never use it daily
- ✓ IAM Users — for humans; permanent credentials
- ✓ IAM Roles — for services; temporary STS credentials
- ✓ Trust Policy + Permissions Policy — how Roles work
- ✓ Least Privilege — grant only what is needed, nothing more

📁 Before Next Class

1. Without looking at notes — explain the difference between an IAM User and an IAM Role in one sentence
2. In the AWS Console: create an IAM User, add it to a Group, and attach a policy — then create an IAM Role for EC2
3. Post on the WhatsApp group: describe one scenario where you would use a Role instead of a User, and why
4. Re-read the three exam traps on— could you explain each one to a classmate from memory?
5. Read the AWS IAM best practices page (link on Slide 15) — identify two practices you will apply in tomorrow's lab



Further Resources & Reading

Go deeper with these official AWS resources:

IAM — What Is IAM?

docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

IAM Roles — How They Work

docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html

IAM Best Practices

docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

AWS STS — Temporary Credentials

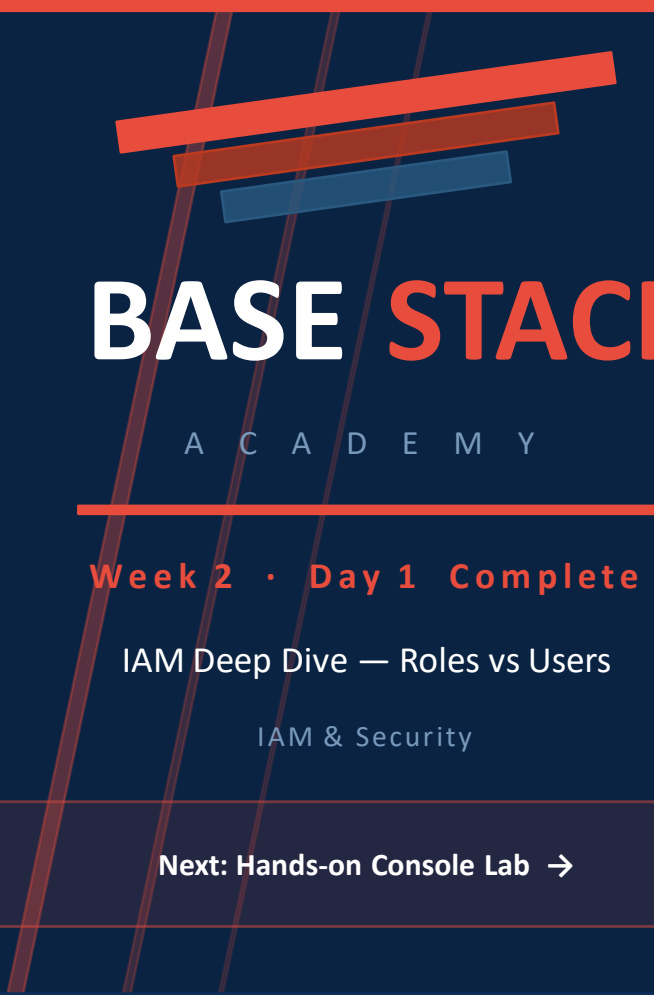
docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

Video — IAM Crash Course (freeCodeCamp / AWS)

youtube.com/watch?v=iF9fs8Rw4Uo (search: AWS IAM Full Course)



Homework: Read at least ONE link above before the next class. Post what you learned on the WhatsApp group.



BASE STACK

A C A D E M Y

Week 2 · Day 1 Complete

IAM Deep Dive — Roles vs Users

IAM & Security

Next: Hands-on Console Lab →